

ROBOTICS · LESSON 1

Servo From a Pot

Turn a knob and a servo arm follows it — the complete input → process → actuate loop every robot runs at its core.

ROBO-001 · ~30 MIN · BEGINNER

1. The one idea

Turn a knob, and a servo arm follows it. You are building the complete **input** → **process** → **actuate** loop that every robot, however complex, runs at its core.

2. Why it matters

A six-axis industrial arm and this one servo run the same fundamental loop: read a desired value, convert it into a command a motor understands, drive the motor. Master it on one servo and one knob, and the rest of robotics is that loop repeated, sensed, and closed. Everything else is detail on top of this.

3. Parts & wiring

- ESP32 dev board
- SG90 micro servo
- B10K potentiometer (linear 10k)
- External 5V supply for the servo (your bench PSU at 5V is perfect) — this is the step beginners skip and then spend an hour debugging; see §7
- Breadboard + jumpers

Wire	From	To	Why
Pot wiper (middle pin)	pot	GPIO34	input-only ADC pin; reads 0–3.3V cleanly and won't conflict with WiFi
Pot outer pins	pot	3V3 and GND	gives the wiper a full 0–3.3V sweep
Servo signal (orange)	servo	GPIO13	PWM-capable pin the servo library can drive

Wire	From	To	Why
Servo power (red)	servo	external 5V, not the ESP32	servos spike current; powering from the board browns it out
Servo ground (brown)	servo	GND — shared with the ESP32's GND	signal is meaningless without a common ground reference

The single most important wire here is the shared ground. A servo powered from a separate supply with no common ground will twitch, ignore commands, or sit dead — and the wiring looks right. Call this out loud; it's invisible to you and invisible to them.

4. Predict (do this before uploading)

Write down your guesses:

- Pot turned fully left → servo angle = ?
- Pot fully right → ?
- Pot centered → ?

Hold onto those numbers. You'll check them in §6.

5. Build it — the sketch

```
#include <ESP32Servo.h>

Servo armServo;           // the servo object we send angles to

const int POT_PIN  = 34;   // ADC pin - pot wiper connects here
const int SERVO_PIN = 13;  // PWM pin - servo signal wire

void setup() {
  Serial.begin(115200);
  armServo.attach(SERVO_PIN); // bind the servo object to its pin
}

void loop() {
  int raw  = analogRead(POT_PIN); // knob position, 0-4095 (12-bit ADC)
  int angle = map(raw, 0, 4095, 0, 180); // scale knob range → servo range
  armServo.write(angle); // command the servo to that angle
  Serial.println(angle); // watch the number as you turn the knob
  delay(15); // brief pause so the servo can keep up
}
```

Three lines in `loop()` are the entire robot: **read, convert, act**. Point that out — it's the whole lesson in three lines, and naming that is what turns code into understanding.

Library: install **ESP32Servo** from the Arduino Library Manager. `analogRead` on the ESP32 returns 0–4095 because the ADC is 12-bit — that's why the `map()` uses 4095, not Arduino's usual 1023. Saying *why* the number is different is the kind of invisible step worth surfacing.

6. Test against your prediction

Open the Serial Monitor at 115200 baud. Turn the knob slowly and watch the number, then watch the arm. Did the extremes match your §4 guesses? If the direction is reversed from what you expected, that's a great teaching moment — flip the `map()` arguments to `(0, 4095, 180, 0)` and ask why that reverses it.

7. What goes wrong (pre-loaded debugging)

- **ESP32 keeps resetting when the servo moves → brownout.** The servo is pulling power from the board. Move servo power to the external 5V supply. (This is the §3 gotcha biting.)

- **Servo twitches or ignores the knob** → **no common ground** between the 5V supply and the ESP32. Tie the grounds together.
- **Arm doesn't move at all** → signal wire on the wrong pin, or `attach()` not called, or you wired power to 3V3 (servos want ~5V).
- **Serial value frozen or random** → if you moved the pot to a different pin, make sure it's an ADC1 pin (32–39). ADC2 pins stop working once WiFi is on — a classic ESP32 trap.

8. Checkpoint

You can now turn a knob and have a motor follow it in real time. That is the complete **sense** → **command** → **actuate** loop of robotics, running on your bench. Every robot you build from here is this loop — repeated across more joints, fed by sensors instead of a knob, and eventually closed so the motor checks whether it actually got where it was told.

9. Push further

- Swap the pot for the HC-SR04 ultrasonic sensor: arm angle tracks distance. Now the input is the world, not your hand — the first taste of autonomy.
- Add a second servo on another PWM pin for pan/tilt. Two loops at once.

These two extensions lead straight toward the rover (two motors + L298N), which is Robotics Lesson 2 or 3.